Design Patterns

ilustrate în proiectul **Kinder**

magazin online pentru copii — Laravel 11

Disciplină	TMPPP – Universitatea Tehnică a Moldovei
Autor	Mihail Chircu
Tip proiect	Proiect de semestru – implementare 12+ design patterns
Stack tehnologic	PHP 8.3 · Laravel 11 · SQLite · Tailwind 3 · Vite
Repository	github.com/mihail-chircu/TMPPP

Sumar al pattern-urilor

Proiectul implementează 15 pattern-uri din catalogul GoF (refactoring.guru/design-patterns/catalog), repartizate astfel: 4 creaționale, 5 structurale și 6 comportamentale. Minimul cerut la disciplină este 12 (4+4+4), deci există surplus.

#	Categorie	Pattern	Unde trăiește în cod
01	Crea■ional	Builder	OrderBuilder
02	Crea■ional	Prototype	Product::duplicate
03	Crea■ional	Factory Method	ShippingMethod → Calculators
04	Crea■ional	Abstract Factory	OrderNotificationFactory (Email/SMS)
05	Structural	Facade	CheckoutFacade
06	Structural	Decorator	Pricing (Base/GiftWrap/Discount/Tax)
07	Structural	Adapter	Payment Adapters + Gateways
08	Structural	Proxy	CachingProductProxy
09	Structural	Composite	Category (arbore)
10	Comportamental	Strategy	ProductSorter + 4 strategii
11	Comportamental	State	OrderState + 5 st■ri
12	Comportamental	Command	CartCommand + Invoker cu Undo
13	Comportamental	Chain of Resp.	OrderValidationPipeline
14	Comportamental	Template Method	OrderExportTemplate
15	Comportamental	Observer	OrderPlaced event + 3 Listeners

I. Pattern-uri creaționale

Pattern-urile creaționale abstractizează modul în care obiectele sunt instanțiate. În proiect le folosim pentru a gestiona construcția comenzilor, clonarea produselor, selectarea calculatoarelor de shipping și producerea notificărilor per canal.

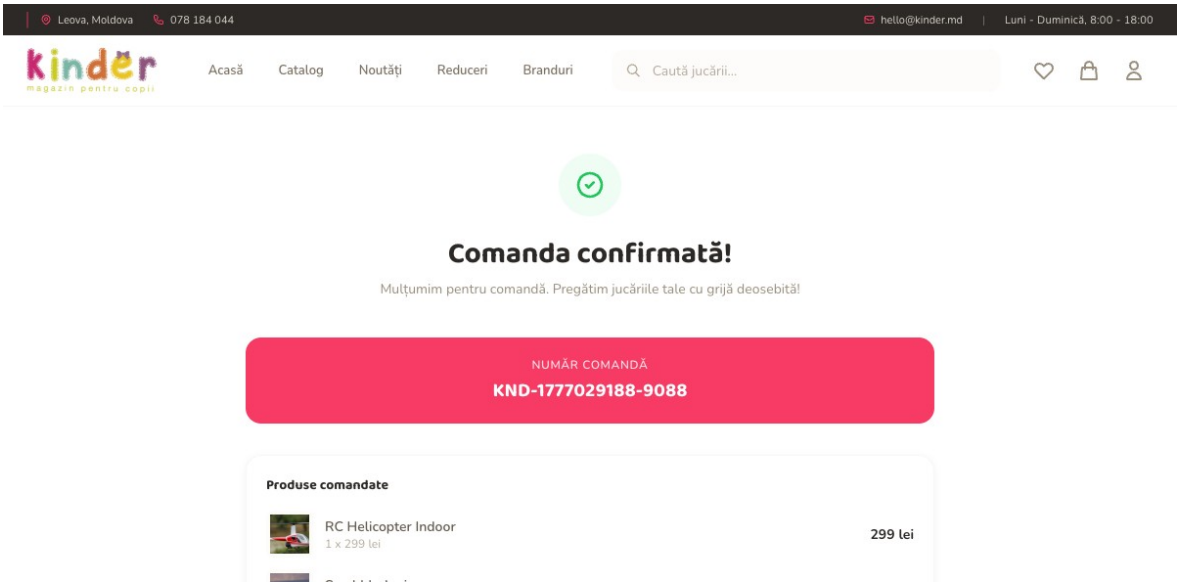
01. Builder

Creațional

Clasificare GoF	Creational – Builder
Problema rezolvată	Entitatea **Order** are peste 15 câmpuri obligatorii (client, adresă, metode de plată și livrare, items, totaluri) și construcția implică logică colaterală (tranzacție DB, decoratori de preț, decrementare stoc, incrementare sales_count). Un constructor masiv sau mai multe overload-uri ar fi fragile.
Componente (roluri)	**Builder**: `OrderBuilder` expune o interfață fluentă (`forUser`, `withCustomer`, `withShipping`, `withPaymentMethod`, `withShippingMethod`, `withGiftWrap`, `withCartItems`, `withNotes`). **Director**: `CheckoutFacade` orchestrează apelurile pe builder. **Product**: modelul Eloquent `Order` cu `OrderItem`-urile asociate.
Fișiere cheie	`app/Services/Order/OrderBuilder.php` `app/Services/Checkout/CheckoutFacade.php`

Cum funcționează în aplicație

La checkout, `CheckoutFacade::processCheckout()` asamblează obiectul Order pas cu pas: setează datele clientului, adresa, metodele, aplică decoratorii de preț prin `PriceCalculator` (BasePrice → GiftWrapDecorator), creează `Order` și `OrderItem`-urile, decrementează stocul și incrementează numărul de vânzări — toate într-o singură tranzacție `DB::transaction`. Builder-ul NU e folosit ca înlocuitor de setters — construcția e reală, complexă și non-trivială, exact ca la carte.



Rezultatul apelului Builder: comandă confirmată cu număr unic, items, total.

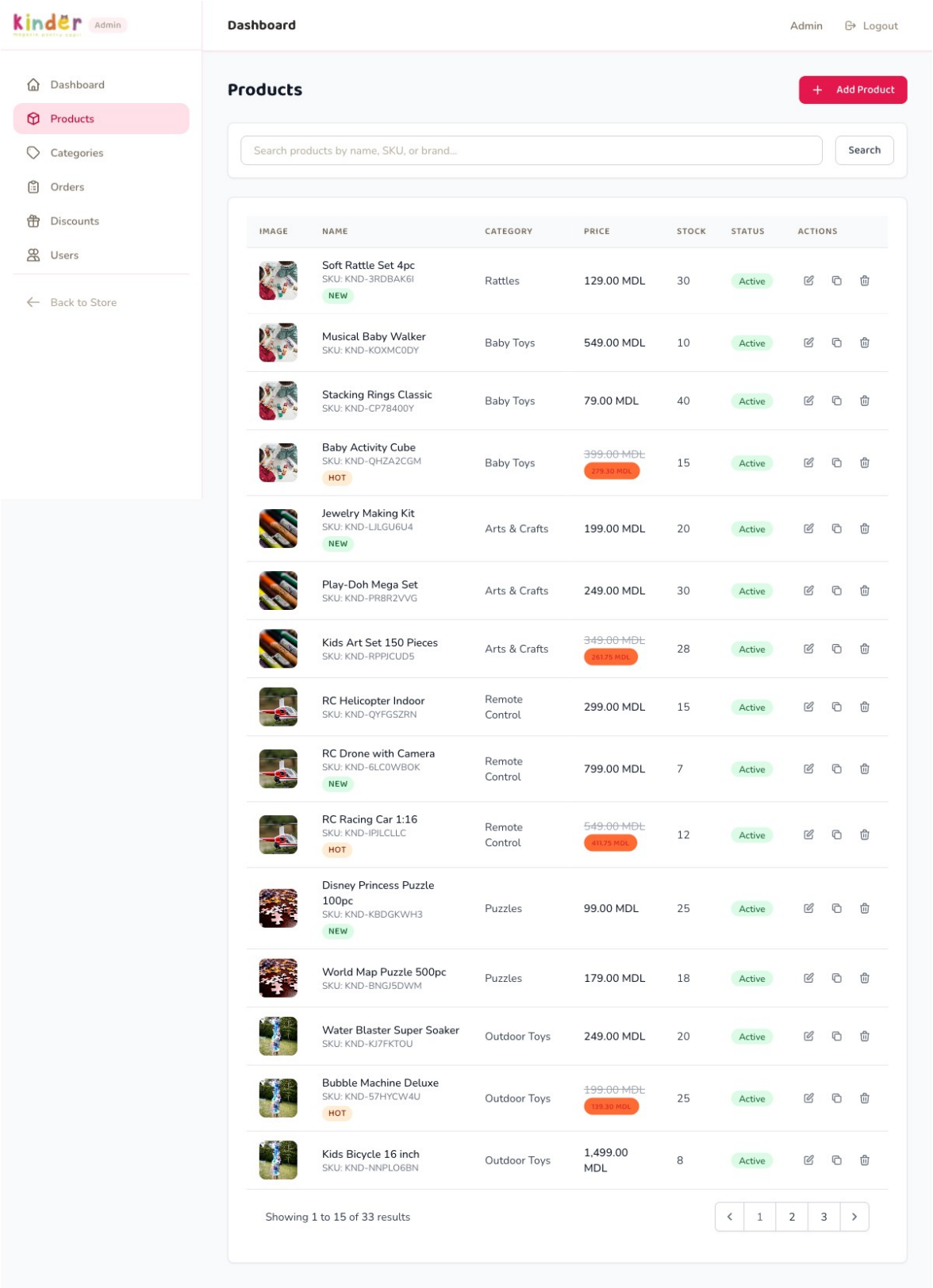
02. Prototype

Creațional

Clasificare GoF	Creational – Prototype
Problema rezolvată	Admin-ul are nevoie să creeze variante ale unui produs existent (ex. aceeași jucărie în altă culoare, cu preț/stoc diferit) fără a cupla codul la clasa concretă a produsului și fără a rescrie manual 20 de câmpuri.
Componente (roluri)	**Prototype interface**: `duplicate(): self` **Concrete Prototype**: `Product::duplicate()` – deep-clone prin `replicate()`, slug și SKU regenerate, imaginile (`ProductImage`) recopiate.
Fișiere cheie	`app/Contracts/Prototype.php` `app/Models/Product.php` (implements Prototype)

Cum funcționează în aplicație

În `AdminProductController::duplicate()`, butonul «Duplicate» apelează `$product->duplicate()`. Clientul nu știe dacă primește un `Product` concret — doar că e un `Prototype`. Copia primește nume „… (Copie)”, slug unic, SKU nou, `views_count` și `sales_count` resetate, și `is_active=false` ca să nu apară public până nu e ajustată de admin.



Pagina Admin → Products. Butonul copy (icon dublu) declanșează Prototype::duplicate.

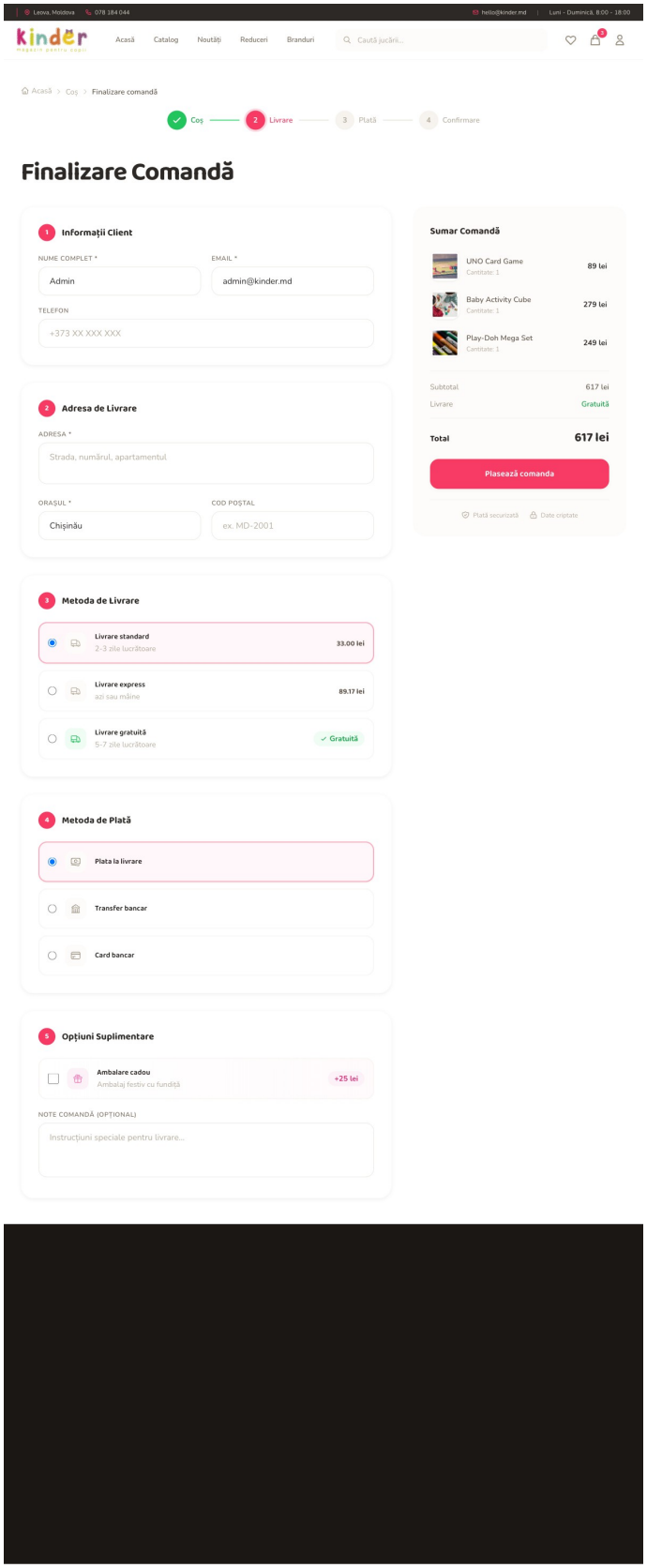
03. Factory Method

Creațional

Clasificare GoF	Creational – Factory Method
Problema rezolvată	Aplicația oferă trei metode de livrare (Standard, Express, Gratuită) cu formule de preț radical diferite: per-item, supratăxă în afara Chișinăului, weekend, asigurare, discount de volum, prag minim. Vrem ca „ce metodă ai ales” să decidă „cum se calculează prețul” fără `if/else`-uri imbricate.
Componente (roluri)	**Creator**: clasa abstractă `ShippingMethod` cu metoda fabrică `createCalculator(): ShippingCalculator`. **ConcreteCreator**: `StandardShipping` / `ExpressShipping` / `FreeShipping`. **Product**: interfața `ShippingCalculator`. **ConcreteProduct**: `StandardCalculator` / `ExpressCalculator` / `FreeCalculator`.
Fișiere cheie	`app/Services/Shipping/ShippingMethod.php` (Creator) `app/Services/Shipping/Methods/*.php` (Concrete Creators) `app/Services/Shipping/Calculators/*.php` (Products) `app/Services/Shipping/ShippingQuote.php` (context)

Cum funcționează în aplicație

`CheckoutFacade` construiește un `ShippingQuote` (total, număr items, oraș) și îl trimite la `ShippingService`. Serviciul întreabă fiecare `ShippingMethod` dacă e eligibil (Free doar peste 500 MDL) și apelează `getShippingCost()`, care intern folosește metoda fabrică. Fiecare calculator are logică reală — Standard: 35 + 3/item suplimentar + 15 surcharge afară din Chișinău – 5 discount >300 MDL; Express: 65 + 6/item + 25 afară din oraș + 20 weekend + asigurare 1 % capped la 40; Free: 0 doar dacă pragul e atins.



Checkout step 3: cele trei metode de livrare cu prețuri calculate dinamic de calculatoare distincte.

04. Abstract Factory

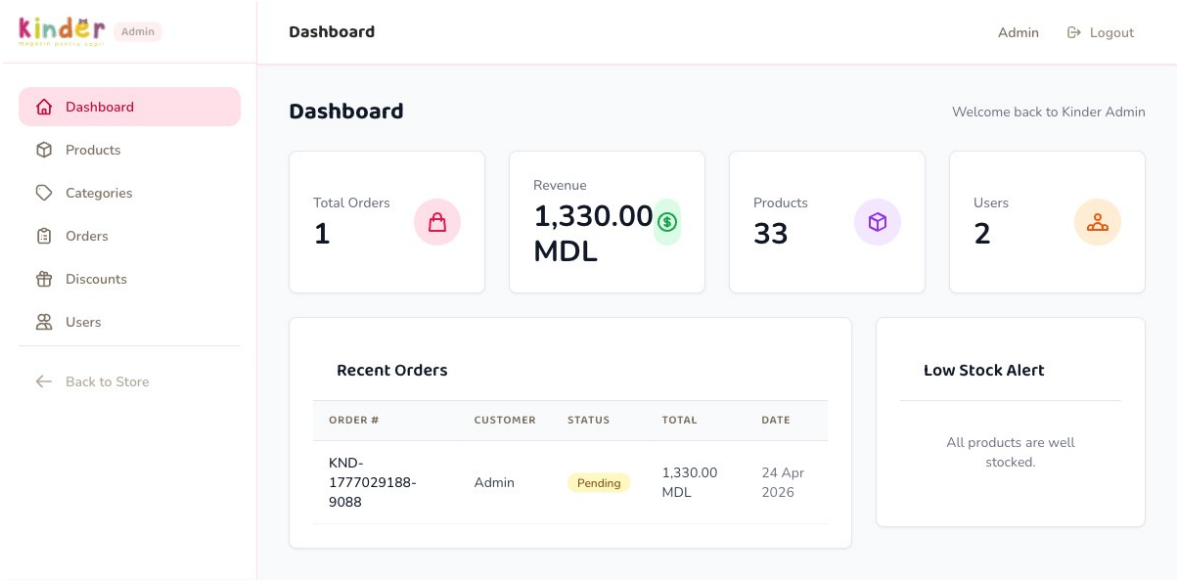
Creațional

Clasificare GoF	Creational – Abstract Factory
Problema rezolvată	Magazinul trimite trei feluri de notificări legate de comandă — confirmare, update status, expediere — pe mai multe canale (Email, SMS). Fiecare canal are constrângeri diferite (email poate fi narativ, SMS trebuie să fie concis). Vrem ca o singură alegere (canalul) să producă o familie consistentă de notificări, fără ca clientul să cunoască implementările concrete.
Componente (roluri)	**AbstractFactory**: `OrderNotificationFactory` (interface cu 3 create*). **ConcreteFactory**: `EmailNotificationFactory`, `SmsNotificationFactory`. **AbstractProduct**: `NotificationContent` (clasă abstractă). **ConcreteProducts**: `OrderConfirmationNotification` (cu orderSummary), `StatusUpdateNotification` (previousStatus/newStatus), `ShippingNotification` (trackingCode/estimatedDelivery).

Fișiere cheie	`app/Notifications/Factory/OrderNotificationFactory.php` (interface)
	`app/Notifications/Factory/{OrderConfirmation,StatusUpdate,Shipping}Notification.php` (Abstract Products)
	`app/Notifications/Channels/Email/EmailNotificationFactory.php`
	`app/Notifications/Channels/Sms/SmsNotificationFactory.php`

Cum funcționează în aplicație

`AppServiceProvider::register()` face bind pe canalul default (Email) către interfață. `OrderNotificationService` primește factory-ul prin DI și îl apelează fără să-i cunoască implementarea. Fiecare `ConcreteFactory` produce toate trei notificările în stilul canalului său — email-urile au text lung și listă de produse; SMS-urile sunt scurte. Cele trei clase concrete de produs sunt distincte (nu doar același DTO cu channel diferit), deci „familia” este reală — exact definiția Abstract Factory.



Observer triggers Abstract Factory: dashboard-ul arată comanda nouă și revenue-ul actualizat în urma evenimentului OrderPlaced.

II. Pattern-uri structurale

Pattern-urile structurale descriu modul în care obiectele și clasele sunt compuse pentru a forma structuri mai mari. În proiect le folosim pentru orchestrare (Facade), compunere dinamică de preț (Decorator), adaptare gateway-uri de plată (Adapter), caching invizibil (Proxy) și ierarhii de categorii (Composite).

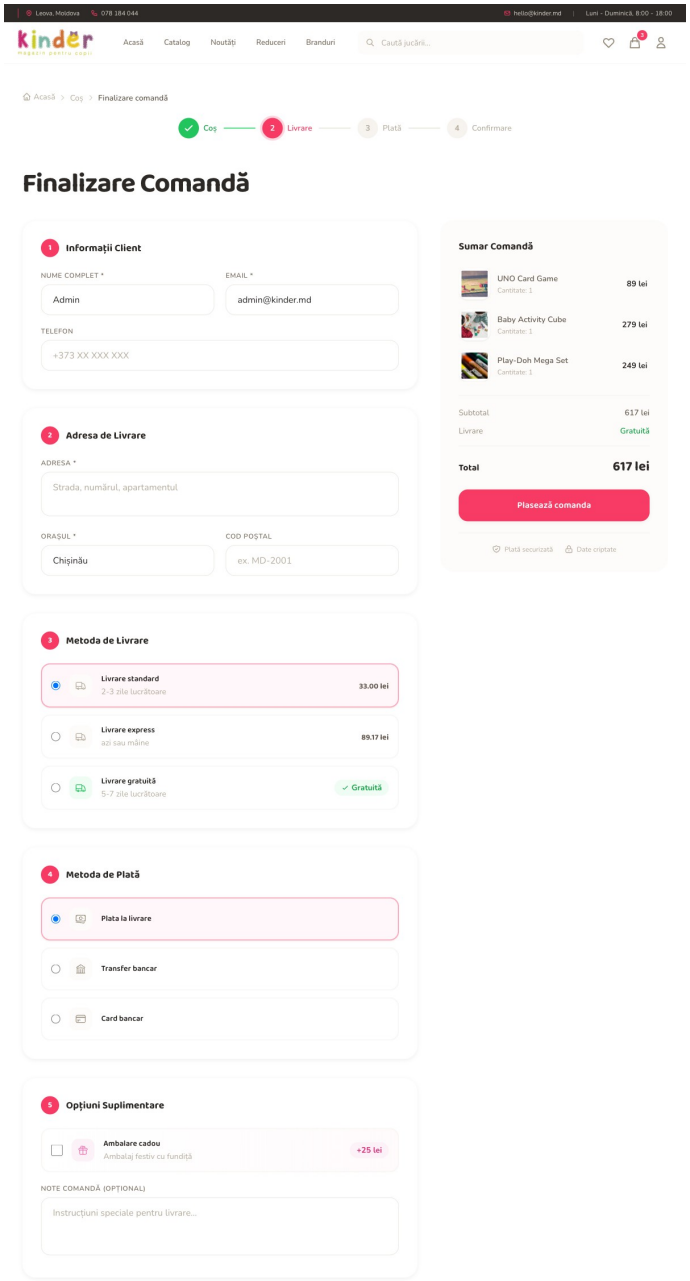
05. Facade

Structural

Clasificare GoF	Structural – Facade
Problema rezolvată	Procesul de checkout implică cel puțin cinci subsisteme: validare (Chain of Responsibility), calcul shipping (Factory Method), construcție comandă (Builder), procesare plată (Adapter) și notificări (Observer + Abstract Factory). Un controller care ar chema fiecare subsistem direct ar fi cuplat la 5 clase concrete și greu de testat.
Componente (roluri)	**Facade**: `CheckoutFacade::processCheckout($data, $cart)`. **Subsisteme ascunse**: `OrderValidationPipeline`, `ShippingService`, `OrderBuilder`, `PaymentService`, `CartCommandInvoker`, `Event::dispatch`. **Client**: `CheckoutController` — un singur apel.
Fișiere cheie	`app/Services/Checkout/CheckoutFacade.php`

Cum funcționează în aplicație

Controller-ul validează form-ul, apoi apelează `$checkoutFacade->processCheckout($validated, $cart)`. Facade-ul: rulează validările prin Chain, construiește `ShippingQuote` și calculează costul via Factory Method, invocă `OrderBuilder` pentru a asambla comanda, curăță coșul și istoricul de comenzi (Command invoker), apelează `PaymentService` care internă folosește Adapter, și dispatch-uiește evenimentul `OrderPlaced` pentru Observers. Tot acest flux e invizibil pentru controller.



Pagina de checkout – orchestrată integral de Facade după submit.

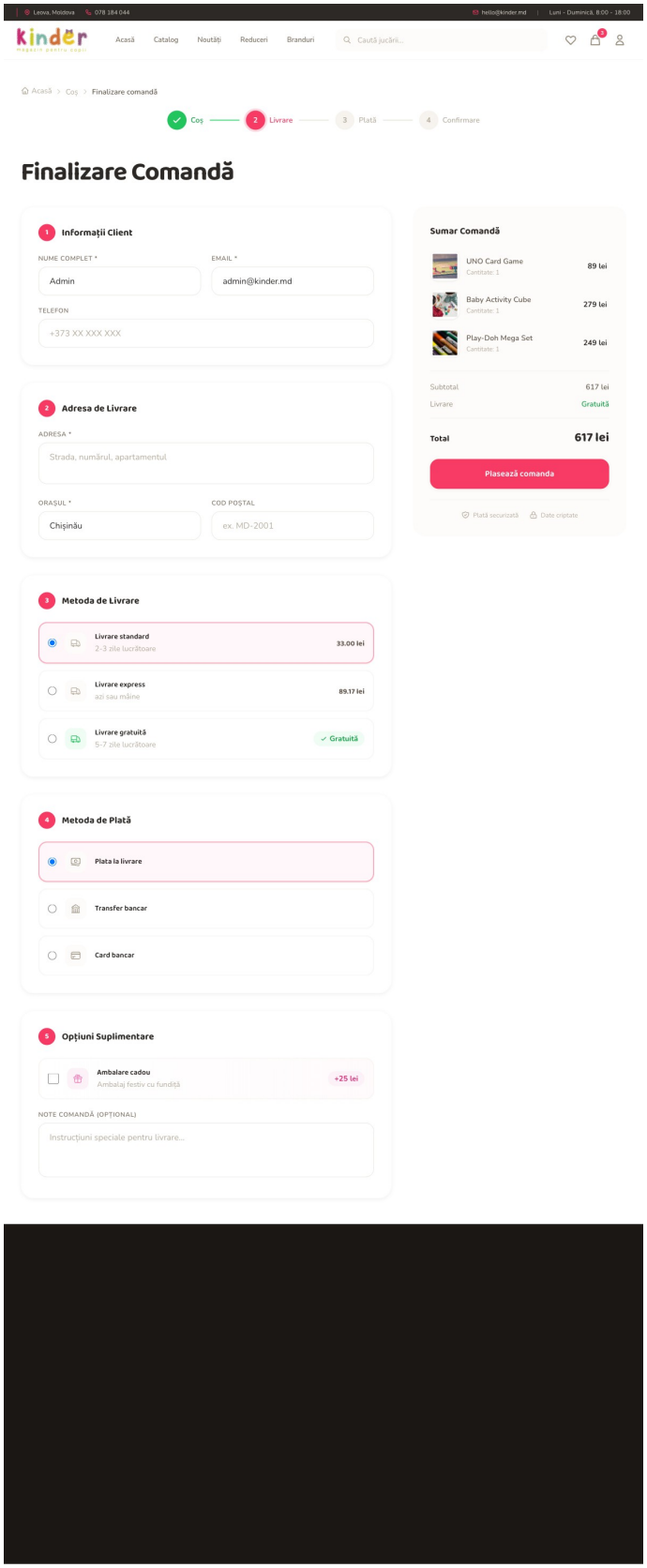
06. Decorator

Structural

Clasificare GoF	Structural – Decorator
Problema rezolvată	Prețul final al unei comenzi depinde de modificatori compozabili: ambalaj cadou (+25 MDL), discount procentual, TVA. Numărul și ordinea modificatorilor se pot schimba (a mâine adăugăm „cupon loialitate” sau „taxa ambalaj eco”). Vrem să adăugăm modificatori fără să atingem clasa de bază.
Componente (roluri)	**Component**: `PriceCalculator::calculate(): float`. **ConcreteComponent**: `BasePrice` — returnează subtotalul. **Decorator abstract**: `PriceDecorator` wrap-uiește un alt `PriceCalculator`. **ConcreteDecorators**: `GiftWrapDecorator` adaugă fee fix, `DiscountDecorator` aplică procent, `TaxDecorator` adaugă TVA.
Fișiere cheie	`app/Services/Pricing/PriceCalculator.php` (interface) `app/Services/Pricing/BasePrice.php` (Component) `app/Services/Pricing/PriceDecorator.php` (abstract) `app/Services/Pricing/{GiftWrap,Discount,Tax}Decorator.php`

Cum funcționează în aplicație

În `OrderBuilder::buildPrice()` construim o „pisălogi” de decoratori: `new GiftWrapDecorator(new BasePrice($subtotal))`. Fiecare strat își adaugă contribuția și forwardează la wrapped. Metoda `getBreakdown()` agregă contribuțiile ca listă de linii pentru chitanță. Adăugarea unui nou modificator nu cere modificări la BasePrice sau la apelanți — doar o clasă nouă care extinde `PriceDecorator`.



Checkout step 5: opțiunea „Ambalare cadou +25 lei” — GiftWrapDecorator aplicat la runtime.

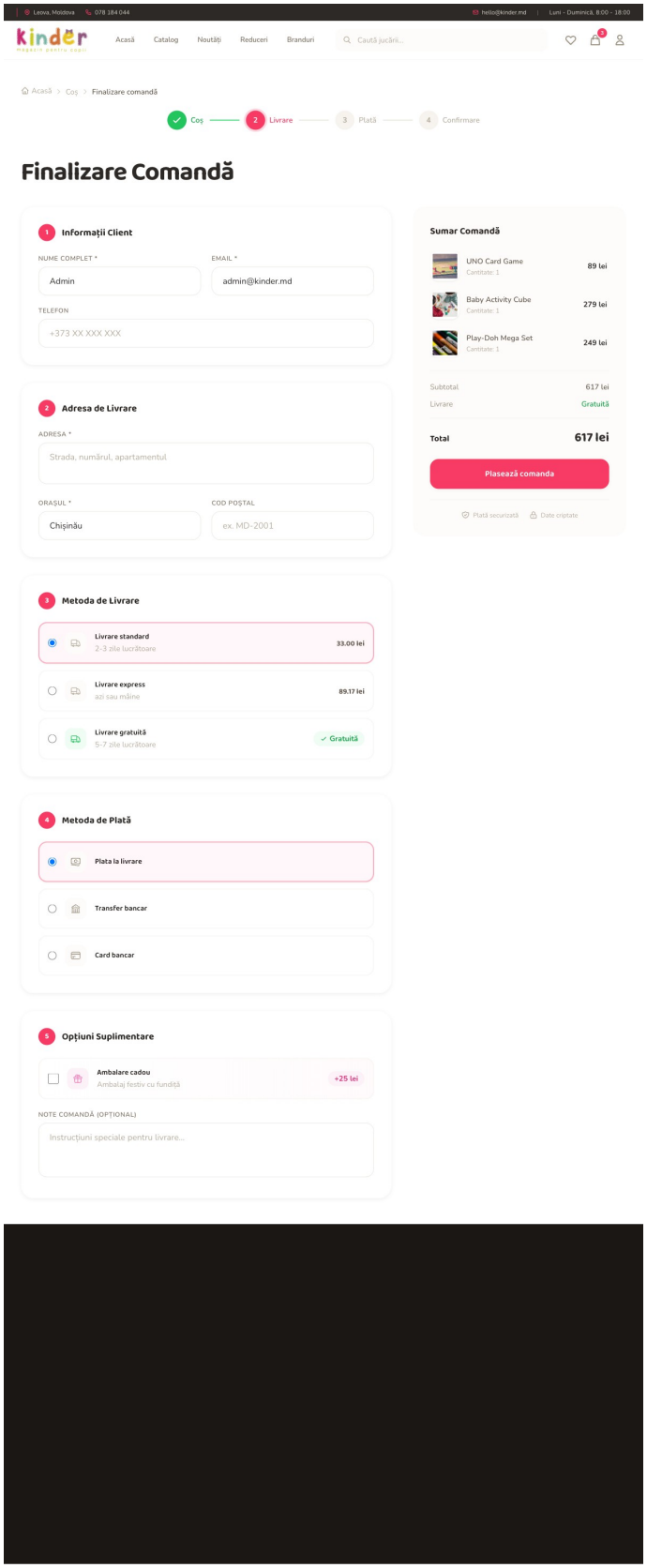
07. Adapter

Structural

Clasificare GoF	Structural – Adapter
Problema rezolvată	Avem de integrat un gateway tip Stripe (sume în *cenți*, currency lowercase, card tokens, răspuns cu `id` / `status` / `amount_captured` / `card.last4`) și un API bancar local (IBAN, SWIFT, state machine `PENDING_CONFIRMATION` → `SETTLED` / `REJECTED`, ISO-8601, error objects). Restul aplicației trebuie să poată procesa plata indiferent de gateway.
Componente (roluri)	**Target**: `PaymentProcessorInterface` – metode `process(Order $o): array` cu shape unificat. **Adaptees**: `StripeGatewayClient`, `LocalBankApiClient` – API-uri străine cu shape-uri diferite. **Adapters**: `CardPaymentAdapter` convertește MDL → cenți, tokenizează cardul, traduce răspunsul Stripe. `BankTransferAdapter` deschide un transfer, interpretează state machine-ul, expune URL-ul de instrucțiuni. **Client**: `PaymentService` – vede doar interfața unificată.
Fișiere cheie	`app/Services/Payment/PaymentProcessorInterface.php` (Target) `app/Services/Payment/Gateways/StripeGatewayClient.php` (Adaptee 1) `app/Services/Payment/Gateways/LocalBankApiClient.php` (Adaptee 2) `app/Services/Payment/Adapters/*.php` (Adapters)

Cum funcționează în aplicație

`PaymentService` deține un dict `code → adapter`. Când `CheckoutFacade` cere `process($order, 'card')`, `CardPaymentAdapter` transformă totalul în cenți, generează `tok_*`, apelează `StripeGatewayClient::createCharge()`, și mapează răspunsul (`status='succeeded'`) la shape-ul nostru `{success, transaction_id, message}`. Aplicația nu știe și nu trebuie să știe că gateway-ul vorbește în altă limbă.



Checkout step 4: cele trei metode de plată expun aceeași interfață – în spate, adaptoarele traduc între API-uri diferite.

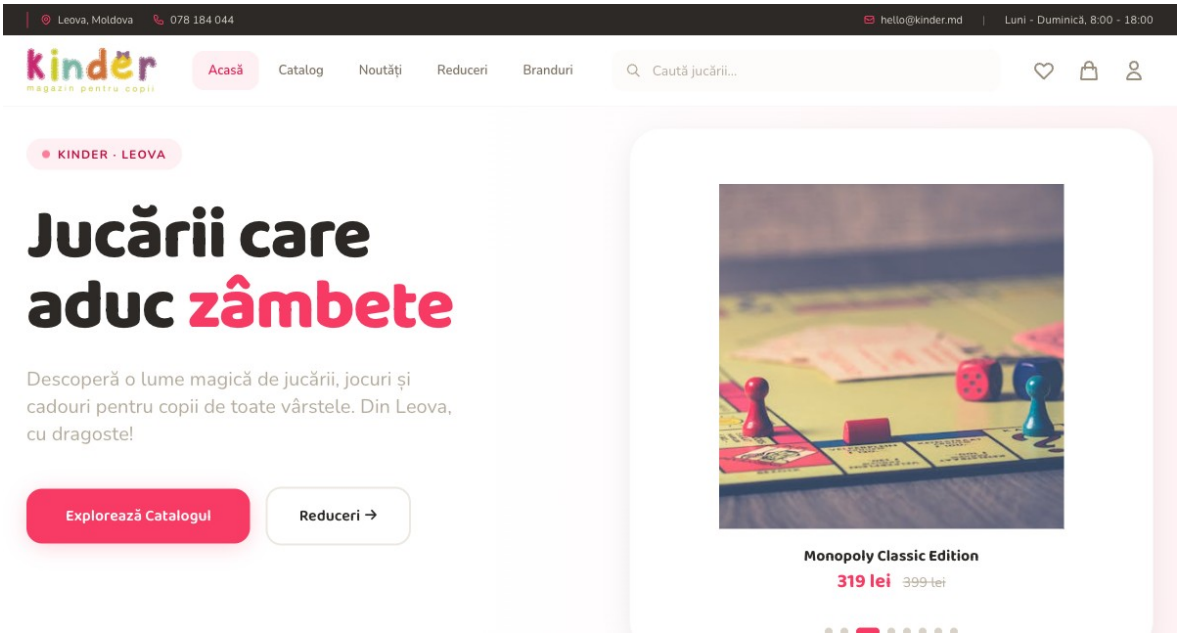
08. Proxy

Structural

Clasificare GoF	Structural – Proxy (Caching Proxy)
Problema rezolvată	Homepage-ul afișează patru secțiuni care fac interogări grele la DB (**featured**, **new**, **sale**, **popular**). Refacem aceste query-uri la fiecare vizită e risipitor. Vrem caching transparent, fără ca HomeController să știe.
Componente (roluri)	**Subject**: `ProductRepositoryInterface – getFeatured / getNew / getSale / getPopular`. **RealSubject**: `ProductRepository` – query-uri Eloquent reale. **Proxy**: `CachingProductProxy` – wrap peste real subject, `Cache::remember(..., TTL 30 min, fn)` pentru fiecare metodă.
Fișiere cheie	`app/Contracts/ProductRepositoryInterface.php` (Subject) `app/Repositories/ProductRepository.php` (RealSubject) `app/Repositories/CachingProductProxy.php` (Proxy)

Cum funcționează în aplicație

`AppServiceProvider::register()` face bind: `ProductRepositoryInterface::class → CachingProductProxy::class`. `HomeController` typehint-uiește interfața și primește proxy-ul prin DI. Clientul apelează `$repo->getFeatured()` ca orice repository — primul apel atinge DB, următoarele 30 de minute vin din cache. A doua zi schimbăm implementarea (ex. `RedisCacheProxy`) fără să atingem HomeController.



Hero homepage + carusel produse „featured” – servite prin CachingProductProxy.

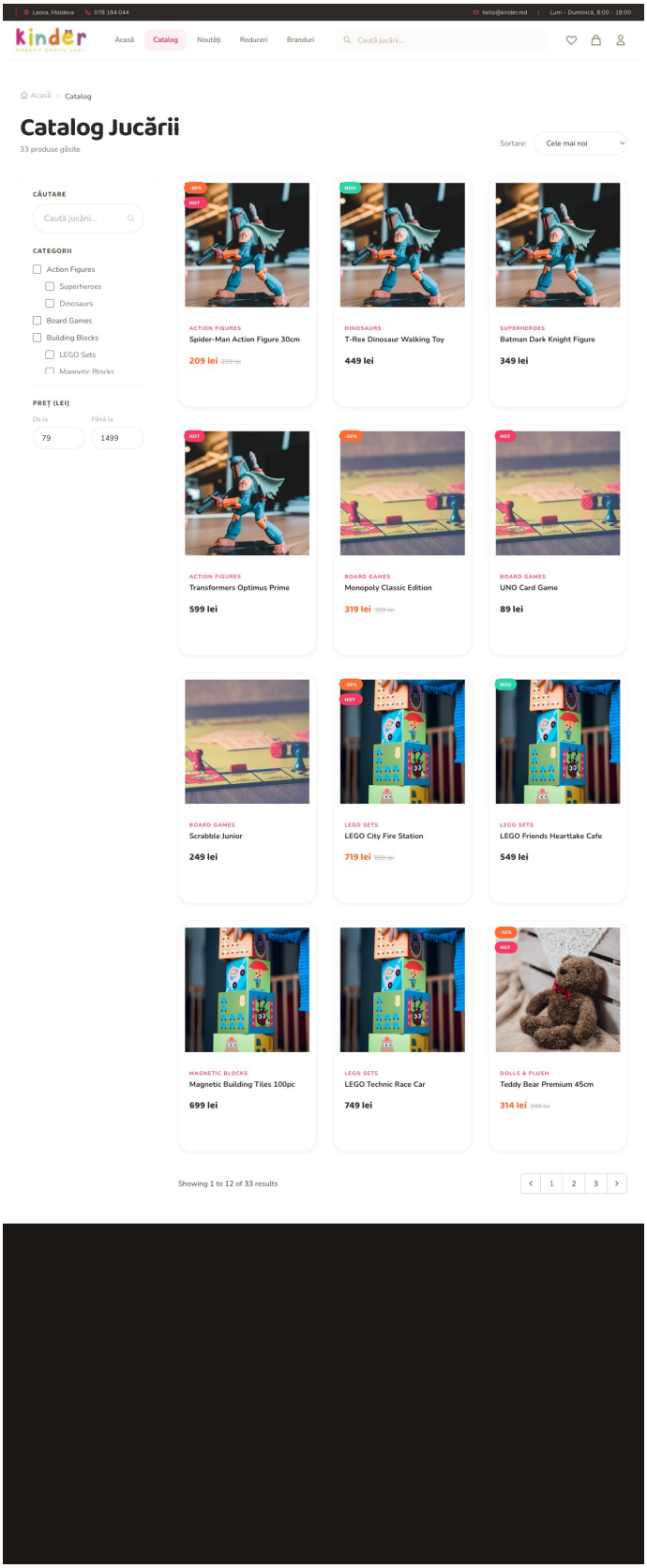
09. Composite

Structural

Clasificare GoF	Structural – Composite
Problema rezolvată	Categoriile formează arbore: **Action Figures** are copiii **Superheroes**, **Dinosaurs**; **Building Blocks** are **LEGO Sets**, **Magnetic Blocks** etc. Vrem să afișăm numărul total de produse pe categorie, agregând și subcategoriile, fără a distinge în client între o frunză și un compozit.
Componente (roluri)	**Component**: `CategoryComponent` – `getName`, `getSlug`, `getProductCount(): int`, `isLeaf()`, `getChildComponents()`. **Leaf & Composite**: aceeași clasă `Category` — o instanță fără copii este Leaf; cu copii este Composite.
Fișiere cheie	`app/Contracts/CategoryComponent.php` (Component) `app/Models/Category.php` implements CategoryComponent (Composite & Leaf)

Cum funcționează în aplicație

`Category::getProductCount()` numără propriile produse, apoi recursiv adună pe fiecare copil activ. Clientul (sidebar-ul de catalog, pagina admin) tratează uniform orice categorie: frunză → doar propriul count; compozit → totalul recursiv. Arborele se construiește din baza de date prin relațiile `parent` / `children`, deci adăugarea unei noi subcategorii nu cere cod nou — doar un rând în DB.



Sidebar catalog – arborele de categorii, cu Composite.getProductCount() recursiv per nod.

III. Pattern-uri comportamentale

Pattern-urile comportamentale descriu algoritmi și distribuția responsabilităților între obiecte. În proiect le folosim pentru sortare interschimbabilă a produselor (Strategy), tranziții de status (State), operații cu undo (Command), validare înlănțuită (Chain of Responsibility), export în formate multiple (Template Method) și reacții post-comandă (Observer).

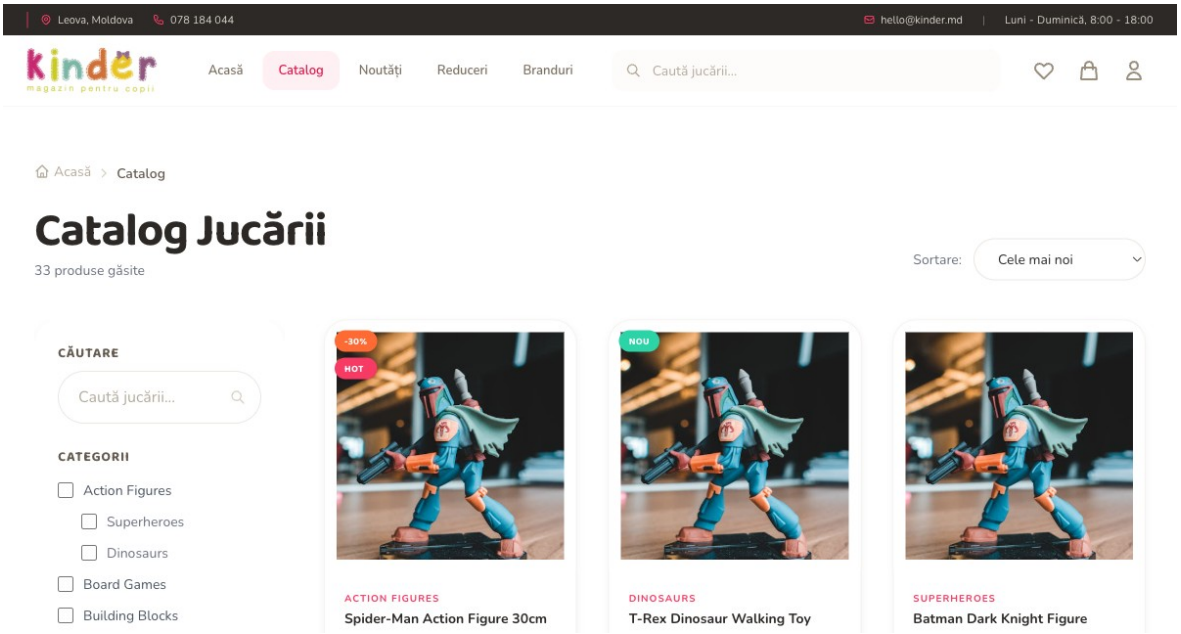
10. Strategy

Comportamental

Clasificare GoF	Behavioral – Strategy
Problema rezolvată	Catalogul de produse oferă patru moduri de sortare: cele mai noi, popularitate, preț ascendent, preț descendent. Vrem să putem adăuga strategii noi (ex. „cele mai bine cotate”, „reducere”) fără să atingem controller-ul.
Componente (roluri)	**Strategy**: `SortingStrategy::apply(Builder $q): Builder`. **ConcreteStrategy**: `NewestStrategy` / `PopularityStrategy` / `PriceAscStrategy` / `PriceDescStrategy`. **Context**: `ProductSorter` – primește cheia din query string, rezolvă strategia, o aplică pe builder-ul Eloquent.
Fișiere cheie	`app/Services/Catalog/SortingStrategy.php` (interface) `app/Services/Catalog/Strategies/*.php` (concrete strategies) `app/Services/Catalog/ProductSorter.php` (context)

Cum funcționează în aplicație

Utilizatorul alege „Sortare” din dropdown-ul catalogului. `CatalogController` trimite cheia la `ProductSorter`. Acesta alege strategia corespunzătoare și aplică un `orderBy` pe query-ul Eloquent, care e apoi paginat. O strategie nouă = o clasă nouă, fără `switch`.



Dropdown „Sortare” – selectarea declanșează strategia corespunzătoare în ProductSorter.

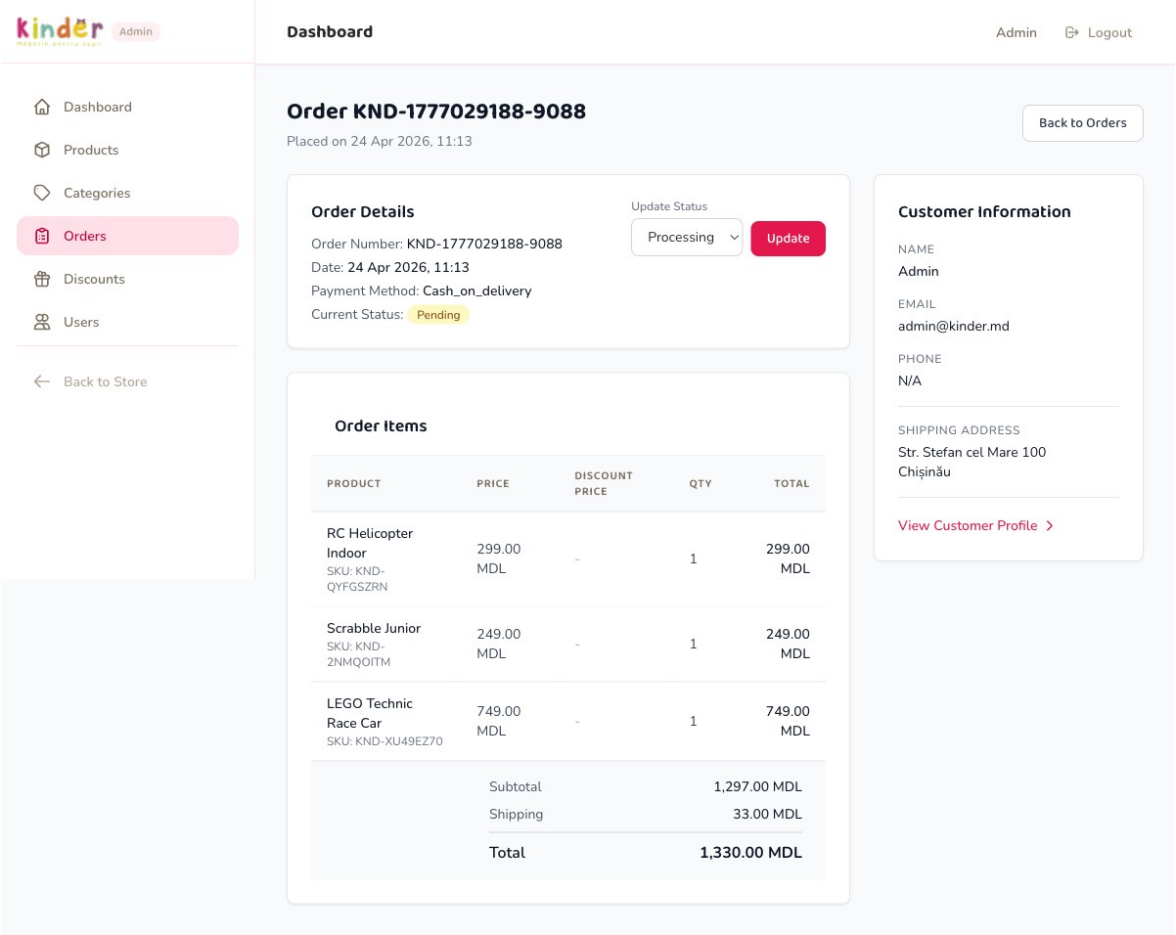
11. State

Comportamental

Clasificare GoF	Behavioral – State
Problema rezolvată	O comandă poate fi în una din cinci stări: *pending*, *processing*, *shipped*, *delivered*, *cancelled*. Fiecare stare are reguli diferite: din *pending* poți merge în *processing* sau *cancelled*, dar nu direct în *delivered*. Un `if/else` gigant pe status ar fi buggy.
Componente (roluri)	**State abstract**: `OrderState` – `canTransitionTo(string $next): bool`, `handle(Order $o): void`, `label(): string`, `next(): array`. **ConcreteStates**: `PendingState`, `ProcessingState`, `ShippedState`, `DeliveredState`, `CancelledState`.
Fișiere cheie	`app/Services/OrderState/OrderState.php` (abstract State) `app/Services/OrderState/States/*.php` (concrete states)

Cum funcționează în aplicație

Când admin-ul schimbă statusul în admin UI, controller-ul cere starea curentă a comenzii, întreabă `canTransitionTo()`, execută `handle()` (ex. `ShippedState` trimite notificarea shipping + setează tracking) și persistă noul status. Adăugarea unei stări noi (ex. *returned*) e o clasă nouă — nu modifici state-urile existente.



Pagina admin order detail – dropdown Update Status arată doar tranzițiile legale.

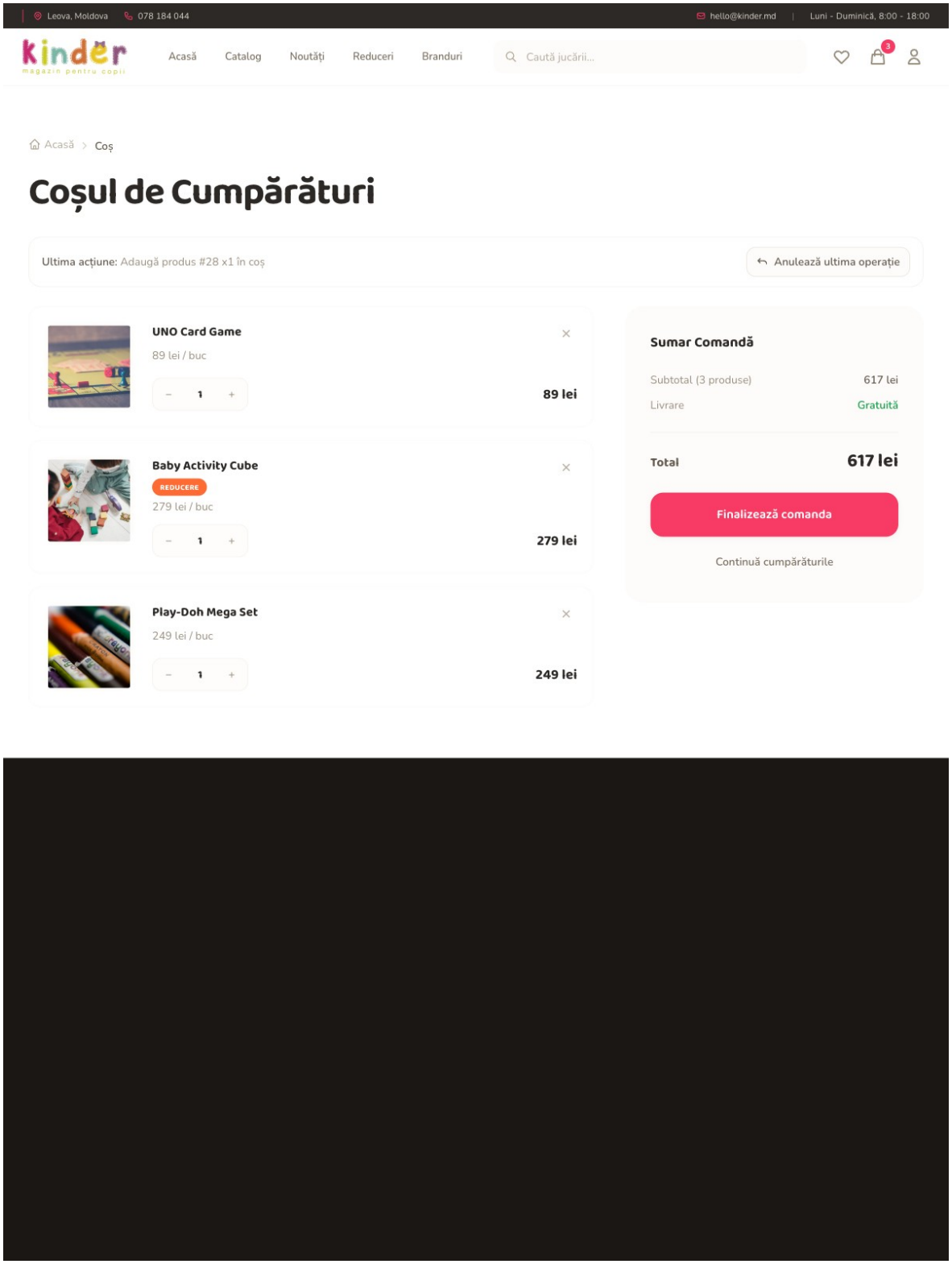
12. Command

Comportamental

Clasificare GoF	Behavioral – Command (cu Undo)
Problema rezolvată	Acțiunile pe coș — adaugă, actualizează cantitate, elimină — au nevoie de reversibilitate („Anulează ultima operație”) și istoric. Metode instantanee pe `CartService` nu permit undo între request-uri HTTP.
Componente (roluri)	**Command**: `CartCommand` – `execute, undo, getDescription`. **ConcreteCommand**: `AddToCartCommand, UpdateQuantityCommand, RemoveFromCartCommand`. Fiecare stochează *snapshot*-ul necesar pentru undo (previous quantity, removed row data). **Invoker**: `CartCommandInvoker` – serializează comanda cu `serialize()` și o păstrează în sesiunea Laravel. **Receiver**: `CartService`, modelele `CartItem`.
Fișiere cheie	`app/Services/Cart/CartCommand.php` (interface) `app/Services/Cart/Commands/{AddToCart,UpdateQuantity,RemoveFromCart}Command.php` `app/Services/Cart/CartCommandInvoker.php` (Invoker, persistat în sesiune)

Cum funcționează în aplicație

La fiecare modificare a coșului, `CartController` creează un Command, îl execută prin invoker și acesta îl salvează serializat în sesiune. În view apare un banner cu ultima acțiune și buton **„Anulează ultima operație”**. Click-ul POST-uiește la `/cart/undo`; invoker-ul dez-serializează ultima comandă și apelează `undo()`. După checkout reușit, `CheckoutFacade` apelează `clearHistory()`, pentru că comenzile devin invalide.



Coș cu banner „Ultima acțiune: Adaugă produs #28 x1 în coș” și buton Undo activ.

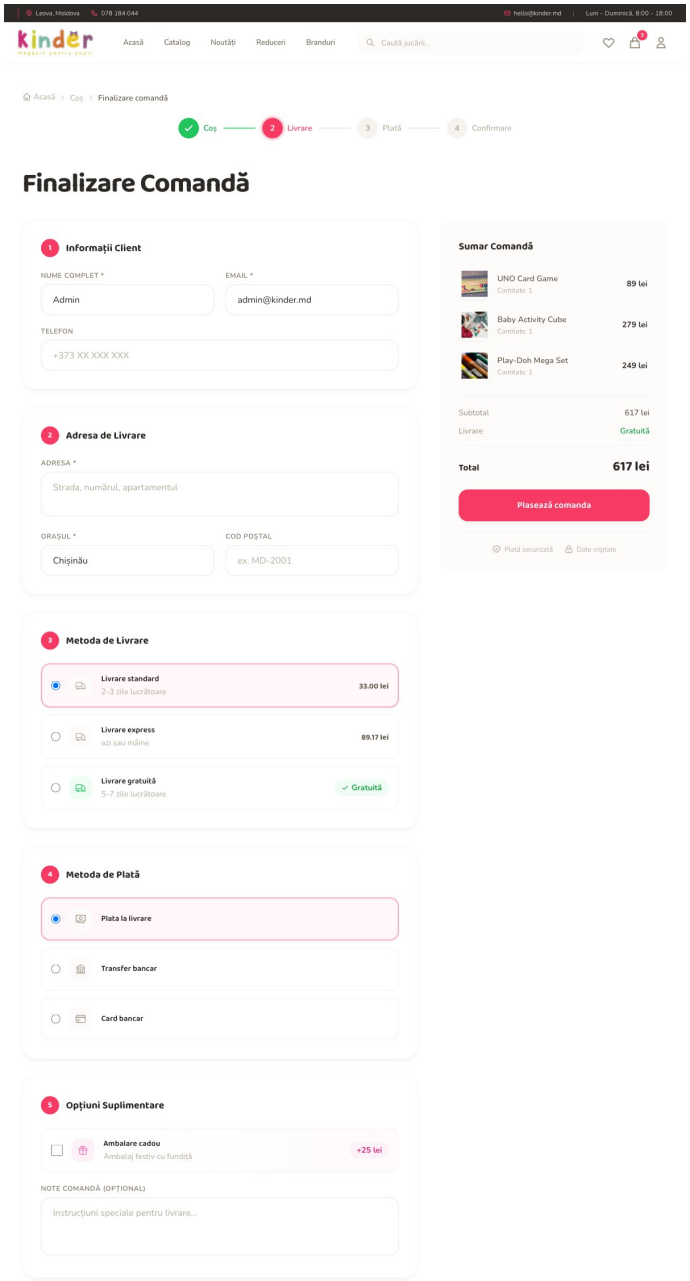
13. Chain of Responsibility

Comportamental

Clasificare GoF	Behavioral – Chain of Responsibility
Problema rezolvată	Înainte de plasarea comenzii trebuie verificate trei lucruri: stoc suficient pentru fiecare item, valoare totală peste minimul configurat, adresă/oraș de livrare valide. Vrem să putem adăuga validatori noi (ex. limită geografică, cod promo) fără să atingem validatorii existenți și fără un `if/else` monolitic.
Componente (roluri)	**Handler**: `OrderValidationHandler` – `setNext`, `handle(array $ctx): ValidationResult`. **ConcreteHandlers**: `StockValidationHandler`, `MinimumOrderHandler`, `ShippingValidationHandler`. **Client**: `OrderValidationPipeline` – construiește lanțul.
Fișiere cheie	`app/Services/Validation/OrderValidationHandler.php` (abstract Handler) `app/Services/Validation/OrderValidationPipeline.php` (client) `app/Services/Validation/Handlers/*.php` (concrete handlers) `app/Services/Validation/ValidationResult.php`

Cum funcționează în aplicație

`CheckoutFacade` trimite contextul la pipeline. Primul handler (Stock) validează; dacă trece, forwardează la următorul (Minimum Order), apoi la Shipping. La prima eroare, lanțul se oprește și întoarce `ValidationResult` cu mesajul. Adăugarea unei noi verificări = o clasă nouă, un singur rând în pipeline.



La submit, toate câmpurile din checkout sunt trecute prin OrderValidationPipeline (Chain of Responsibility).

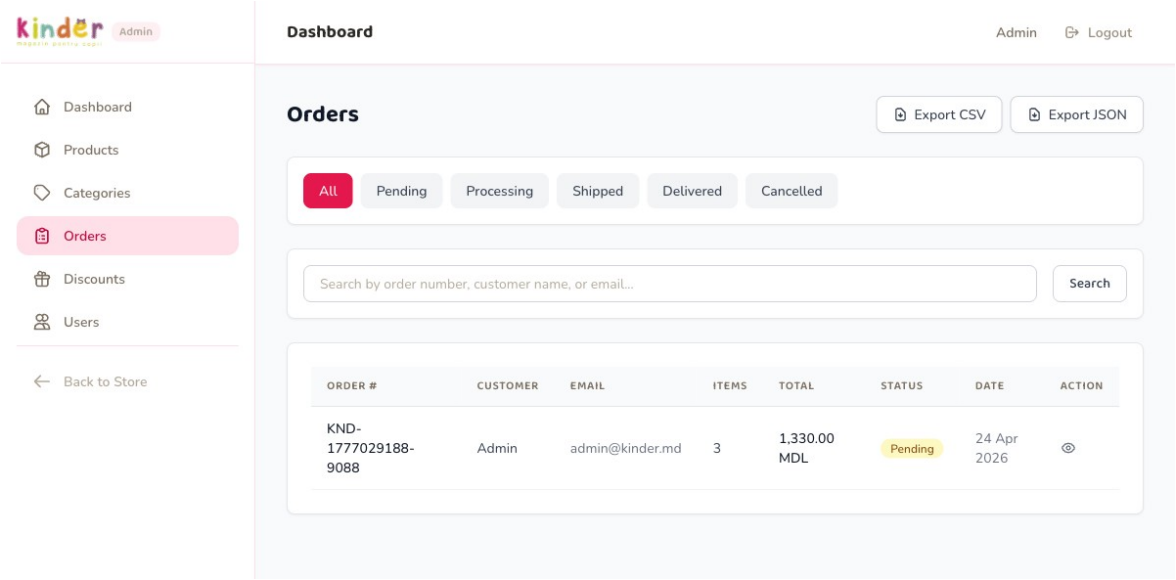
14. Template Method

Comportamental

Clasificare GoF	Behavioral – Template Method
Problema rezolvată	Admin-ul are nevoie să exporte lista de comenzi în formate multiple (CSV, JSON, mâine poate Excel). Algoritmul de export e același: header → iterare rânduri → footer. Doar formatul fiecărei părți diferă.
Componente (roluri)	**AbstractClass**: `OrderExportTemplate` cu metoda template `final export(Collection $orders): string` care apelează `getHeader → formatOrder*... → getFooter`. **ConcreteClass**: `CsvOrderExport`, `JsonOrderExport` – override-uri pentru fiecare hook + MIME type + file extension.
Fișiere cheie	`app/Services/Export/OrderExportTemplate.php` (abstract class) `app/Services/Export/{Csv,Json}OrderExport.php`

Cum funcționează în aplicație

`AdminOrderController::export($format)` alege clasa concretă, apelează `export()` pe colecția de comenzi și forțează download cu `Content-Type`-ul și extensia potrivite. Un format nou = o clasă nouă; metoda template rămâne neatinsă (declarată `final` pentru a bloca override-uri accidentale).



Pagina admin Orders – butoanele Export CSV / Export JSON mapate direct la ConcreteClass-uri.

15. Observer

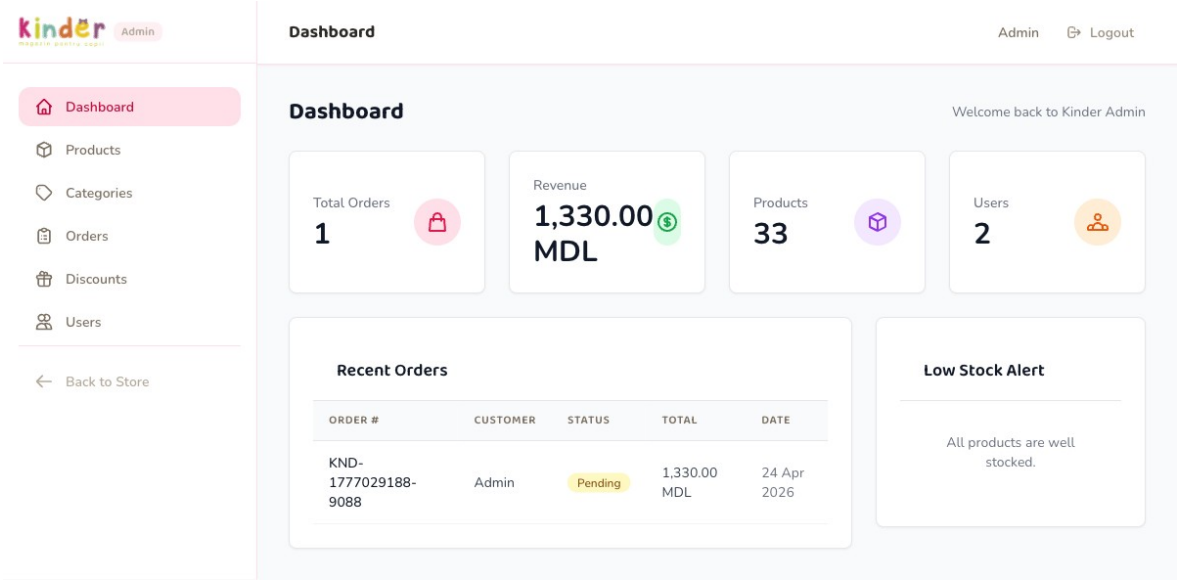
Comportamental

Clasificare GoF	Behavioral – Observer
Problema rezolvată	Când se plasează o comandă, trebuie să se întâmple trei lucruri independente: (1) se trimite email de confirmare, (2) se actualizează statisticile din dashboard, (3) se loghează evenimentul. Le-am putea apela secvențial din `CheckoutFacade`, dar atunci fiecare nou side-effect ar trebui adăugat acolo.
Componente (roluri)	**Subject / Event**: `OrderPlaced` – poartă instanța `Order`. **Observers / Listeners**: `SendOrderConfirmation` (folosește Abstract Factory), `UpdateDashboardStats`, `LogOrderActivity`. **Event dispatcher**: Laravel `Event` facade.

Fișiere cheie	`app/Events/OrderPlaced.php` (Subject / Event)
	`app/Listeners/SendOrderConfirmation.php`
	`app/Listeners/UpdateDashboardStats.php`
	`app/Listeners/LogOrderActivity.php`
	`app/Providers/AppServiceProvider.php` (registration)

Cum funcționează în aplicație

`CheckoutFacade` apelează `event(new OrderPlaced($order))` după ce comanda e persistată. `AppServiceProvider::boot()` a înregistrat toți ascultătorii. Laravel îi invocă sincron în ordine: email confirmare prin factory-ul injectat, incrementare stats din cache/DB, log structurat. Adăugarea unui observer nou (ex. „trimite în Slack”) = o clasă nouă + un rând în `boot()` — **CheckoutFacade nu se atinge**.



Dashboard admin: „Total Orders 1 · Revenue 1 330 MDL" – Listener-ul UpdateDashboardStats a reacționat imediat la event.

Concluzie

Proiectul Kinder integrează 15 pattern-uri din catalogul GoF, fiecare aplicat unei probleme reale din domeniul e-commerce, nu inventată pentru „a bifa”. Alegerea pattern-urilor urmărește principiile SOLID: extensibilitate fără modificare (Open/Closed prin Strategy, Factory Method, Decorator), inversiune de dependințe (Abstract Factory, Adapter, Proxy toate ataşate pe interfețe), separarea responsabilităților (Facade, Chain of Responsibility, Command).

Cele patru fixuri recente (pricing real în Factory Method, produse distincte în Abstract Factory, adaptare efectivă peste gateway-uri mock în Adapter, UI vizibil pentru Undo în Command) au transformat structurile teoretice în exemple potrivite și realiste — exact criteriile cerute la disciplină.

Repository: github.com/mihail-chircu/TMPPP